

Calcolo dei cicli fondamentali in un grafo

14 maggio 2026

In queste note si descrivono due metodi per trovare i cicli fondamentali di un grafo planare connesso. Dato un grafo planare connesso $G(V, E)$, i due metodi produrranno una lista di $k = |E| - |V| + 1$ cicli: nel caso del primo metodo questi non saranno necessariamente di peso minimo, mentre nel caso del secondo metodo lo saranno.

1 Metodo basato su DFS, cicli non necessariamente minimi

Si calcola l'albero DFS $T = \text{dfs}(G)$ ed il coalbero $C = G \setminus T$. Ciascun lato del coalbero, se reinserito nell'albero DFS, richiude un ciclo del grafo originale. Si procede dunque come segue:

1. Per ogni lato $(u, v) \in C$ si calcola il percorso tra u e v in T . Il percorso deve necessariamente esistere.
2. Durante la visita di T si memorizzano tutti i nodi attraversati a partire da u , una volta raggiunto v si aggiunge il percorso alla lista dei cicli trovati.
3. Si itera fino ad esaurire gli archi di C .

L'algoritmo di ricerca del percorso tra u e v può essere qualcosa simile all'Algoritmo 1

Algorithm 1: Finding a path between u and v

Input: DFS tree T ; Nodes u, v

Output: Path between u and v in T

```
findpath ( $T, u, v$ )  
    visited[ $u$ ] = true;  
    path.push( $u$ );  
    if  $u == v$  then  
         $\perp$  return true  
    for  $n \in \text{neighbours}(u)$  do  
        if not visited[ $n$ ] then  
            if findpath( $T, n, v$ ) then  
                 $\perp$  return true  
        path.pop();  
    return false
```

2 Algoritmo di De Pina per la base di cicli minimi

Sia dato un grafo planare connesso $G(V, E)$. L'algoritmo di De Pina usa l'algebra lineare su vettori di valori booleani $S \in \{0, 1\}^m$, con $m = |E|$, per costruire la base di cicli minimi di G .

Anche in questo caso si calcolano l'albero DFS $T = \text{dfs}(G)$ ed il coalbero $C = G \setminus T$. Siano inoltre gli archi (u, v) numerati secondo l'ordine lessicografico assumendo che $u < v$.

Siano S, P vettori di booleani (ovvero di valori 0 oppure 1) di lunghezza N . Il loro prodotto scalare è definito come

$$\langle S, P \rangle := \sum_{i=1}^N S_i \cdot P_i \pmod{2}.$$

Il risultato sarà quindi 0 se S e P hanno un numero pari di 1 in posizioni corrispondenti, mentre sarà 1 se hanno un numero dispari di 1 in posizioni corrispondenti. Sia inoltre $\ominus$ la differenza simmetrica tra vettori di booleani:

$$(S \ominus P)_i := \text{XOR}(S_i, P_i) \quad \forall i \in \{1, \dots, N\}$$

Si inizializzano k vettori booleani S_i di lunghezza m . Per ogni arco $e \in G$ presente anche nel coalbero, S_i sarà uguale a 1 nella posizione corrispondente all'indice di e , mentre sarà uguale a zero in tutte le altre posizioni. Fare attenzione che gli archi del coalbero devono essere esattamente k .

Siano inoltre C_j dei vettori booleani rappresentanti dei cicli: l'elemento in posizione i -esima del vettore sarà uguale a 1 se l'arco i -esimo fa parte del ciclo j -esimo e zero altrimenti. Vettori di questo tipo si chiamano *vettori d'incidenza*.

L'algoritmo di De Pina è il seguente:

Algorithm 2: De Pina

Input: Graph G ; Vectors S_i
Output: Minimum cycle basis $\mathcal{B}$
for $i \in 1, \dots, k$ **do**
 $C_i = \text{find minimal cycle such that } \langle C_i, S_i \rangle = 1$
 $\mathcal{B} = \mathcal{B} \cup C_i$
 for $j \in i + 1, \dots, k$ **do**
 if $\langle C_i, S_j \rangle = 1$ **then**
 $S_j = S_j \ominus S_i$
return $\{C_1, \dots, C_k\}$

Rimane non specificato cosa voglia dire “find minimal cycle such that $\langle C_i, S_i \rangle = 1$ ”, e forse questa è la parte più delicata da comprendere ed implementare: dobbiamo essenzialmente trovare un ciclo che abbia un numero dispari di archi tra quelli “attivi” in S_i .

Per fare questo è necessario effettuare un lifting di $G(V, E)$ ottenendo un grafo ausiliario $G'(V', E')$. Il lifting è costruito nel seguente modo:

1. Si prendono tutti i vertici $v \in G$ e si duplicano, ottenendo v^+ e v^- .
2. Per ogni lato $(u, v) \in G$:
 - se (u, v) è attivo in S_i allora si aggiunge a G' gli archi (u^+, v^-) e (u^-, v^+) .
 - in caso contrario si aggiunge a G' gli archi (u^+, v^+) e (u^-, v^-) .
3. Per ogni vertice $v \in G$ si calcola il cammino minimo, usando per esempio Dijkstra, tra v^- e v^+ in G' .
4. I cammini trovati includeranno nodi “positivi” e nodi “negativi”, ovvero saranno composti da archi (u^α, v^β) con $\alpha, \beta \in \{+, -\}$. Per ogni cammino trovato si costruisce quindi un vettore d'incidenza C_μ di lunghezza $|E|$ in cui, per ogni occorrenza di (u^α, v^β) , si incrementa modulo 2 l'elemento relativo all'arco (u, v) .

5. Di tutti i C_μ trovati se ne conserva solo uno tra quelli con il più piccolo numero di elementi 1, questo sarà il vettore C_i cercato.

Si noti che nell'algoritmo sopra ci sono due “minimi”: prima, per ogni v , si cerca il cammino minimo tra v^+ e v^- , quindi si troverà un certo numero di cammini aventi differenti lunghezze. Di questi cammini, se ne tiene uno tra quelli di lunghezza minore.